Blekinges Tekniska Högskola

Guidelines

March 16, 2026

1 Foreword

This are meant only as a guideline and not as complete explanations.

2 Test-Driven Development

2.1 Introduction

Test-Driven Development (TDD) is an iterative software development method. This method recommends writing tests first, and was officially codified in the book by Beck [1]. TDD Have three phases, Red,Green and Refactor.

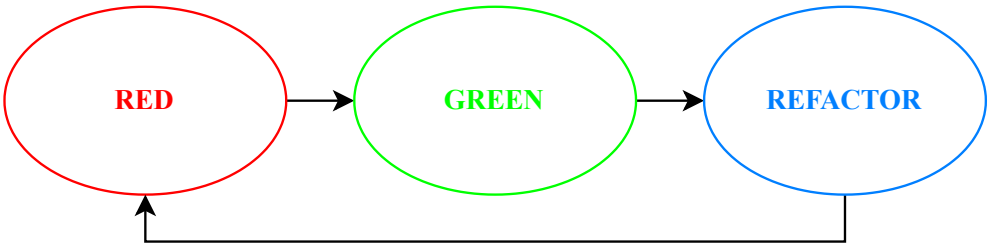

Figure 1: TDD cycle

Note: This section serves as a framework for integrating Large Language Models (LLMs) into the TDD workflow, rather than a general introduction to TDD principles.

2.1.1 Red phase (Human Written)

In the red phase a failing test is written. This test should represent a small functionality of the code and **you have to** watch it fail. In many languages for example like **JAVA** this will give you a compilation fail, it is okay to write boilerplate code to make sure the test runs and **fails.** Notice that writing a test first makes you think about the functionality. There are some **common pitfalls** in the Red phase for example:

1. **Writing too much:** Do not write tests for the entire feature at once, instead focus on an edge case or small functionality.

2. **Testing Implementation, not behaviour:** Focus on what the code outputs, not how it calculates it.

2.1.2 Green phase (LLM generated)

Only after you **see red** it is allowed to begin the Green phase. In this phase you make the test pass as quickly as possible. This phase is only about survival and not writing clean, optimized or beautiful code. Generate the minimum code to make the test pass. In this phase you are allowed to

generate dirty code, by dirty code we mean code that have hardcoded values or duplications. It is important that the Green phase is short and that **you see the test turn green.** Prompt the LLM to write the minimum code to only pass the test and nothing else. An example prompt could be

"Here is a failing test [paste test]. Write the absolute minimum Java code to make this pass".

2.1.3 Refactor phase (Human and LLM refactored)

The refactor phase is where you take your dirty code that passes the test and make it high quality. Notice that you are cleaning the code while the tests stay green. If the test turns red, you've broken something and must undo or fix it immediately. In this phase maintainability, readability, and performance are prioritized and you could do small changes without asking the LLM. **Adding new features is forbidden.** Here you identify code smells and inefficiencies and fix them one by one while the tests stay green. Here you could prompt

"The code now passes. Refactor this code to remove duplication and improve readability without changing its behaviour. Do not add new features."

After going through all three phases you are allowed to do this again meaning you can go to the Red phase to add features.

2.2 Laws

In chapter 5 of the famous book The clean coder by Uncle bob [2], he mentions three laws of TDD and they are the following Law 1-3. **We have added the fourth law to ensure that the laws cover the whole of TDD prompting**

- **Core law 1**: You are not allowed to write any production code until you have first written a failing unit test.
- **Core law 2**: You are not allowed to write more of a unit test than is sufficient to fail—and not compiling is failing.
- **Core law 3**: You are not allowed to write more production code that is sufficient to pass the currently failing unit test.
- **Additional law 4**: You are not allowed to accept LLM-generated code that contains functionality not explicitly covered by your current test suite.

3 Prompt Engineering

3.1 Few-Shot

Few-shot prompting is a technique where you give the model a few examples of what you want before asking it to perform the actual task.

Think of it like showing someone how to do something by demonstration. Instead of just explaining what you want in words, you show 2-3 examples of the input-output pattern you're looking for, then present your real question. When you just give instructions, the model has to guess what you mean or what output you want. But when you show actual examples, there's no guessing—the model can see exactly the length, tone, structure, and level of detail you want.

- **Example: Without Few-shot**

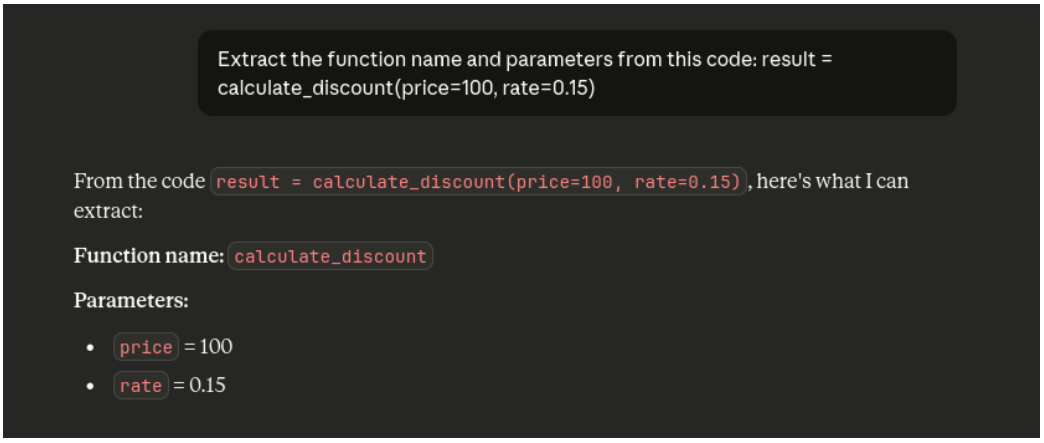

Figure 2: Prompting without Few-Shot

This is a normal prompt without few-shot. The LLM gave us what we wanted but maybe not in the correct format.

- **Example: With Few-shot**

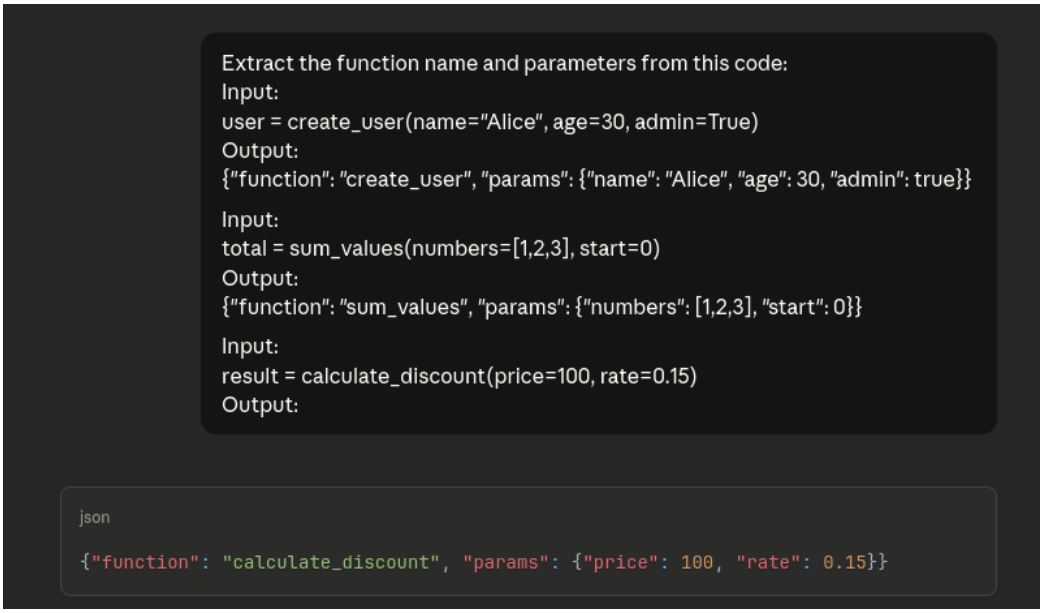

Figure 3: Prompting with Few-Shot

As we can see in the example, when using few-shot Claude 4.5 gave us the wanted output from the start. Often one or two examples is enough to get the expected output.

3.2 Specify code quality and requirements

Specifying code quality requirements means clearly defining the standards you want the LLM to follow when generating code. Instead of just asking for working code, you guide the model on things like coding style, naming conventions, error handling, documentation, modularity, and performance[3]. This turns vague requests into precise instructions. It might seem implied but doing it correctly and actually taking the time to do the prompt with details the code becomes much clearer.

- **Example: Without code specific requirements**

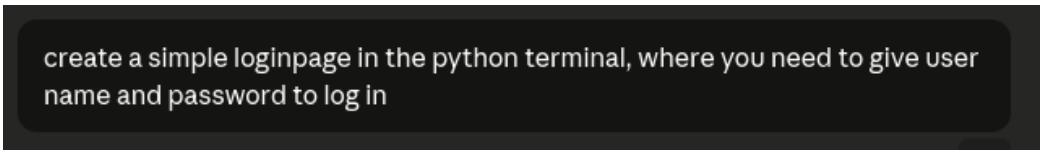

Figure 4: No requirements

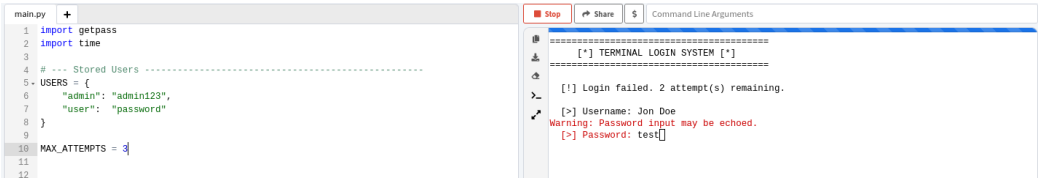

Figure 5: running code

As we can see, running the code works, we can try with a password and user name and it says if we failed and counts number of tries. But there is no sanitation of data and everything is stored directly in the code.

- **Example: With code specific requirements**

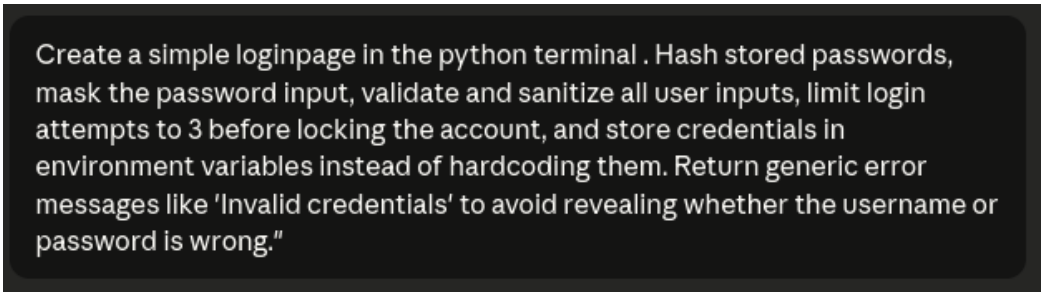

Figure 6: With requirements

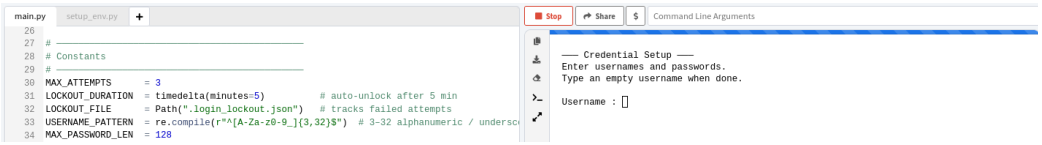

Figure 7: running code

When adding requirements and clear quality control the LLM generates a more specific configuration as expected. This might seem as common knowledge but when prompting, you can never be too specific. With just these extra details the code now made two different files, one for creating a user and storing the hashed data in a json file. So the login page does not know anything about the "real" user name and password.

3.3 Persona

Persona prompting is when you tell the LLM to role play as a specific role prior to the question. This is useful because you can change the LLM's

answer based on the persona it inherits. For example you can ask it to act as a doctor before your question to get a more medical answer or as a teacher to get a more explanatory answer. This also works when programming, by giving the LLM the instruction that its an experienced programmer. This will make the LLM think more when generating code because it now has a higher standard.

- **Example: Without Persona**

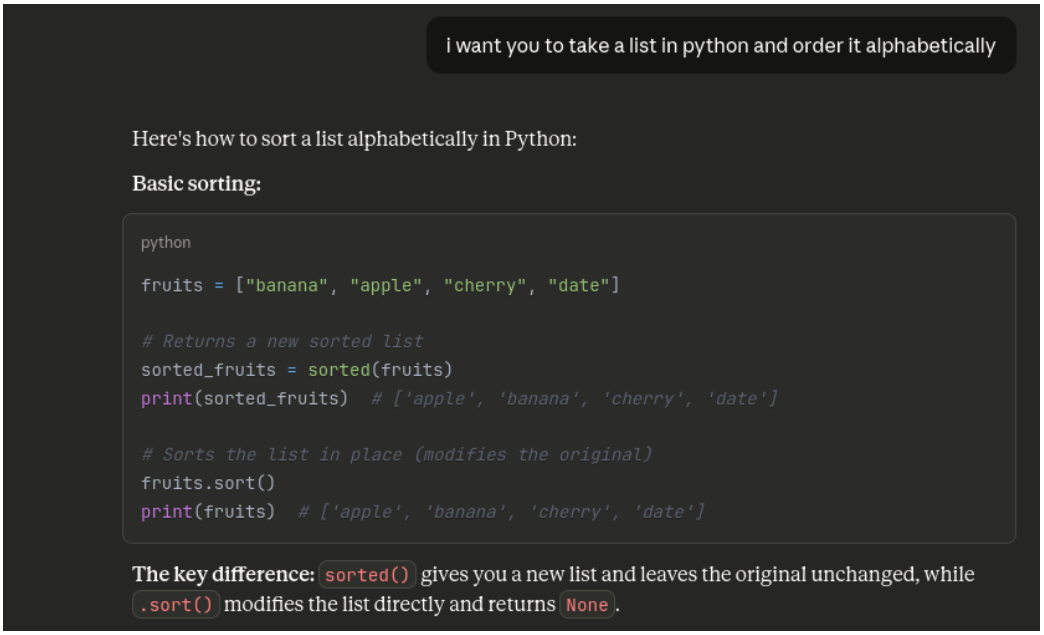

Figure 8: Prompt without persona

we get a simple solution for the prompt. But not much more than that.

- **Example: With Persona**

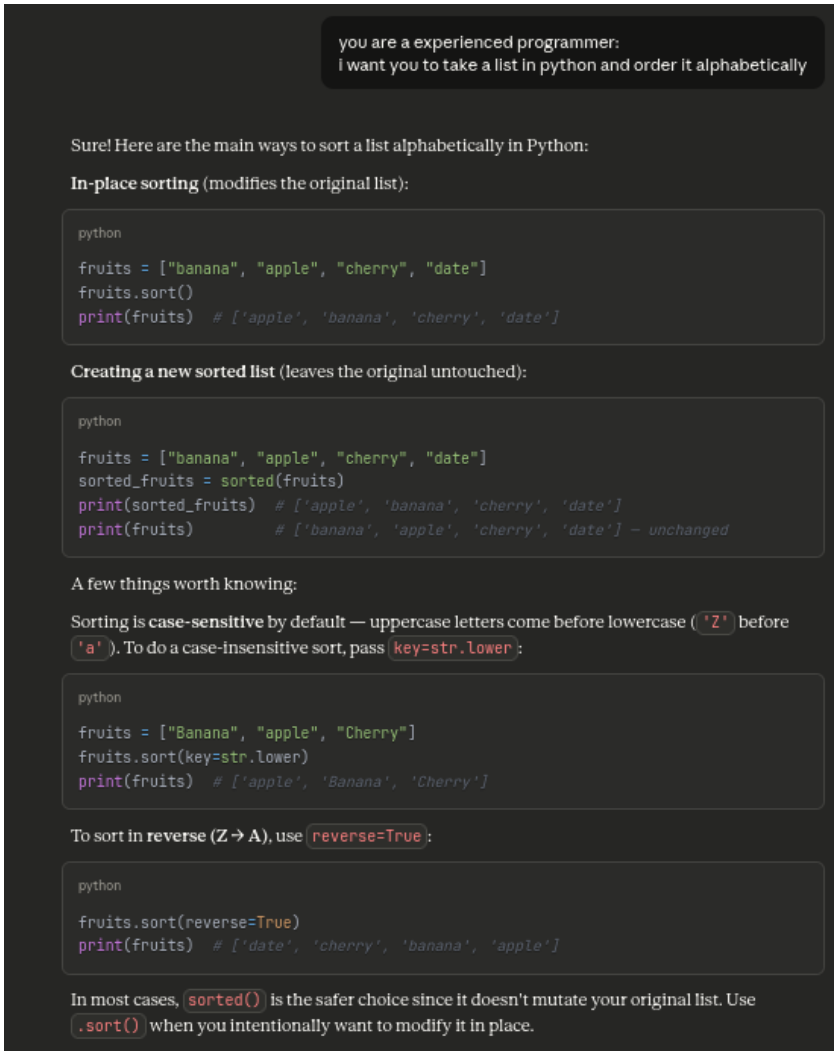

Figure 9: Prompt with persona

When using persona the LLM gives more information, but it also provides different examples, eg. how to reverse sort, and that sort is case sensitive and therefore provide a solution for that case. Its hard to demonstrate the effect of persona when using small examples. But when promoting bigger tasks persona may help to get cleaner and more reliable code.

3.4 Dual-Layer System-User Prompts

Dual-Layer System–User Prompts are a prompting structure where instructions are split into two roles:

- **System layer:** defines how the model should behave.
- **User layer:** defines what the model should do.

The idea is to separate how the model should behave from what task it should perform, so the output is more consistent, controllable, and reliable[4]. Dual layer is a more advanced version of persona and requirement prompting. Dual-layer System-User prompts is one of the most accurate and precise methods to use when writing to a LLM [4].

3.4.1 Example of prompts:

- Example 1

```
System:
You are an expert AI instructor.
Explain concepts clearly and briefly.
Use bullet points when possible.

User:
What is a dual-layer prompt?
```

- Example 2

```
System:
You are a senior software engineer.
Write clean, production-ready code.
Follow best practices and proper naming.
Add concise comments where helpful.
Do not include explanations outside code blocks.

User:
Generate a Python function that reads a text
   file and returns the 10 most frequent words
```

3.5 Combining the different methods

Each method solves a different problem, and in order to get the best result you can combine the methods and in so cover more ground.

Persona prompting may set the tone and perspective of the generated result, but it doesn't guarantee that you get the expected format. Few-shot examples show the model exactly what kind of output you expect, but

without a persona or quality requirements the code might not live up to a specific standard.

There are studies showing that Dual-Layer System-User prompts gives the best result compared to few-shot or persona. But combining the different methods such as few-shot, persona and specific requirements give similar result as Dual-layer [4].

3.5.1 Examples of prompts:

- **Writing a Python function to validate emails**

```
You are a senior backend developer (persona).
Here are two examples of how validation
   functions should be structured in our project
    (few-shot):

Example 1: def validate_age(age): ...
Example 2: def validate_username(username): ...

Now write a function to validate email addresses
   . The code must include error handling,
   docstrings, and unit tests (code quality).
```

- **Creating a user authentication function**

```
You are a cybersecurity-focused backend
   developer (persona). Here are two examples of
    how functions should be structured in our
   project (few-shot):
Example 1:
def hash_password(password):
    # Hashes the password
    ...
Example 2:
def check_token(token):
    # Validates the token and returns user data
    ...
Now write a function that authenticates a user
   by checking their username and password
   against a database. The code must include
   input sanitization, proper error messages,
   password hashing, and logging (code quality).
```

3.6 Best practises

3.6.1 Do's

- **Be Specific:** Specify exact requirements, not vague descriptions
- **Give Context:** Provide background about the codebase, patterns, and conventions
- **Request Tests:** Ask for unit tests, integration tests, or example usage
- **Specify Patterns:** Mention design patterns, architectural patterns, or coding standards
- **Iterate:** Refine prompts based on initial results

3.6.2 Don'ts

- **Don't Assume:** Don't assume the AI knows your structure.
- **Don't Skip Validation:** Always request input validation, sanitization and error handling
- **Don't Ignore Security:** Explicitly request security best practices
- **Don't Skip Documentation:** Ask for comments and documentation

References

[1] K. Beck, *Test-Driven Development: By Example* (The Addison-Wesley Signature Series), 20. printing. Boston: Addison-Wesley, 2015, 220 pp.

[2] R. C. Martin, *The Clean Coder: A Code of Conduct for Professional Programmers*, 1st ed. Upper Saddle River, NJ: Prentice Hall, 2011.

[3] R. Deshmukh, R. Raut, M. Bhavsar, S. Gurav, and Y. Patil, "Optimizing human-ai interaction: Innovations in prompt engineering," 2025. DOI: 10.1109/IDCIOT64235.2025.10914815.

[4] M. Li, E. Nie, H. Huang, X. Lv, and G. Zhou, "Dual-layer prompt ensembles: Leveraging system- and user-level instructions for robust llm-based query expansion and rank fusion," *Information Fusion*, 2026.